MDTS4214_Assignment_7_709

Saurabh Mishra

2026-03-25

Problem Set 7: Shrinkage Methods

We would be working with the Hitters dataset available in the ISLR library. We have to predict the Salary based on all available variables. First, we make sure that we delete all rows with one or more missing values.

1. We split the data set into a training set and a test set randomly, keeping 100 observations in the test set and the remaining in the training set.

```
#install.packages("glmnet")

library(ISLR)

## Warning: package 'ISLR' was built under R version 4.5.2

library(glmnet)

## Warning: package 'glmnet' was built under R version 4.5.3

## Loading required package: Matrix

## Warning: package 'Matrix' was built under R version 4.5.2

## Loaded glmnet 4.1-10

data(Hitters)

# Remove missing values
Hitters = na.omit(Hitters)

dim(Hitters)

## [1] 263  20

set.seed(123)

test_index = sample(1:nrow(Hitters), 100)

test_data = Hitters[test_index, ]
train_data = Hitters[-test_index, ]

dim(train_data)

## [1] 163  20
```

```r
dim(test_data)

## [1] 100  20

# Create model matrices (needed for glmnet)
x_train = model.matrix(Salary ~ ., train_data)[, -1]
y_train = train_data$Salary

x_test = model.matrix(Salary ~ ., test_data)[, -1]
y_test = test_data$Salary
```

2. We train the model using least squares and report the train MSE and test MSE.

```r
lm_model = lm(Salary ~ ., data = train_data)
summary(lm_model)

##
## Call:
## lm(formula = Salary ~ ., data = train_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -809.76 -182.13  -54.69  138.36 1837.14
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  212.67449  129.87001   1.638 0.103705
## AtBat         -2.04895    0.87954  -2.330 0.021229 *
## Hits           5.85495    3.15579   1.855 0.065612 .
## HmRun          5.43622    9.22155   0.590 0.556448
## Runs          -2.49915    4.03131  -0.620 0.536288
## RBI            2.32756    3.84573   0.605 0.545984
## Walks          6.40986    2.41415   2.655 0.008827 **
## Years         -8.30051   17.14327  -0.484 0.628996
## CAtBat         0.01569    0.19354   0.081 0.935512
## CHits         -0.54632    1.02039  -0.535 0.593204
## CHmRun        -2.38189    2.49628  -0.954 0.341604
## CRuns          1.56320    1.05024   1.488 0.138840
## CRBI           1.26993    1.02979   1.233 0.219524
## CWalks        -0.95946    0.45825  -2.094 0.038049 *
## LeagueN       44.32184   98.13766   0.452 0.652220
## DivisionW   -149.54532   56.87996  -2.629 0.009496 **
## PutOuts        0.41589    0.11170   3.723 0.000282 ***
## Assists        0.43589    0.28787   1.514 0.132176
## Errors        -5.50162    5.83866  -0.942 0.347642
## NewLeagueN   -15.79562   97.46148  -0.162 0.871479
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 346.5 on 143 degrees of freedom
```

```
## Multiple R-squared:  0.4782, Adjusted R-squared:  0.4088
## F-statistic: 6.897 on 19 and 143 DF,  p-value: 1.144e-12
```

```r
# Predictions
# Predictions
train_pred_lm = predict(lm_model, newdata = train_data)
test_pred_lm  = predict(lm_model, newdata = test_data)

# MSE function
mse = function(y, yhat){
  mean((y - yhat)^2, na.rm = TRUE)
}

# Compute MSE
train_mse_lm = mse(train_data$Salary, train_pred_lm)
test_mse_lm  = mse(test_data$Salary, test_pred_lm)

train_mse_lm
```

```
## [1] 105334.2
```

```r
test_mse_lm
```

```
## [1] 87803.91
```

3.We fit ridge regression models on the training set, with grid values of λ. We report the following:

(i) the dimension of the coefficient matrix so obtained.

(ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ. We comment on the output.

(iii) Test MSE corresponding to these two choices of λ.

```r
lambda_grid = 10^seq(10, -2, length = 100)

ridge_model = glmnet(x_train, y_train, alpha = 0, lambda = lambda_grid)
```

(i) Dimension of Coefficient Matrix

```r
dim(coef(ridge_model))
```

```
## [1]  20 100
```

Coefficients for 50th and 60th λ

```r
coef_50 = coef(ridge_model)[, 50]
coef_60 = coef(ridge_model)[, 60]

coef_50
```

```
##   (Intercept)         AtBat          Hits         HmRun          Runs
## 438.509969440   0.029684297   0.110523230   0.560867468   0.178502584
##           RBI         Walks         Years        CAtBat         CHits
```

```
##   0.252163513   0.238337160   1.002466622   0.002796464   0.010194612
##        CHmRun         CRuns          CRBI        CWalks       LeagueN
##   0.077846378   0.020089803   0.021957346   0.021866228   0.432252998
##     DivisionW       PutOuts       Assists        Errors    NewLeagueN
##  -6.916280599   0.018294166   0.002998934  -0.082552703   0.741567242
```

```
coef_60
```

```
##  (Intercept)        AtBat         Hits        HmRun         Runs
RBI
## 105.63996577   0.08310078   0.51246876   2.09635144   0.60993278
1.14623958
##        Walks        Years       CAtBat        CHits       CHmRun
CRuns
##   1.01677160   2.77366963   0.01043635   0.04276159   0.27032634
0.08012067
##         CRBI       CWalks      LeagueN    DivisionW      PutOuts
Assists
##   0.09127173   0.05840151  18.35350369 -62.46410535   0.14784491
0.04002532
##       Errors   NewLeagueN
##  -0.86723778  12.21693330
```

(ii). L2 Norm of Coefficients

```
l2_norm = function(beta){
  sqrt(sum(beta^2))
}

l2_50 = l2_norm(coef_50)
l2_60 = l2_norm(coef_60)

l2_50
```

```
## [1] 438.5671
```

```
l2_60
```

```
## [1] 124.7542
```

(iii). Test MSE for 50th and 60th λ

```
# Predictions
ridge_pred_50 = predict(ridge_model, s = lambda_grid[50], newx = x_test)
ridge_pred_60 = predict(ridge_model, s = lambda_grid[60], newx = x_test)

# Test MSE
mse_50 = mse(y_test, ridge_pred_50)
mse_60 = mse(y_test, ridge_pred_60)

mse_50
```

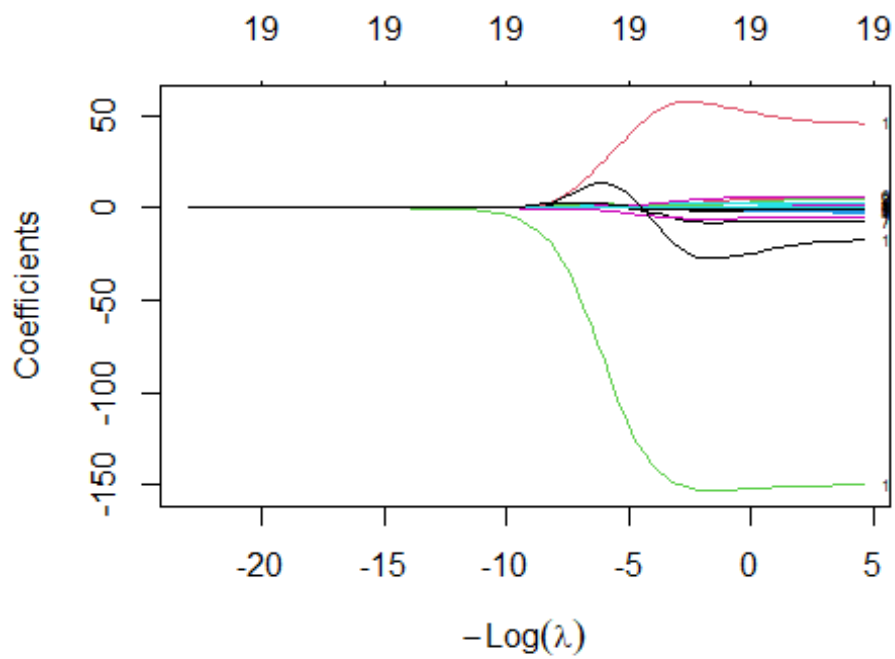
```
## [1] 167303.1
```

```
mse_60
```

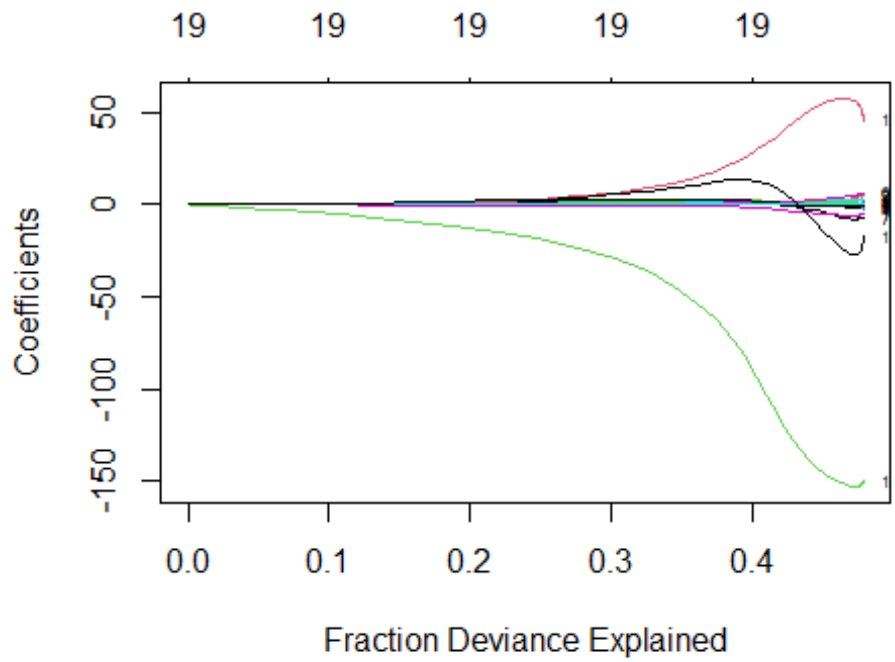
```
## [1] 97935.56
```

4. Make a line plot (for all the predictors) of the standardized coefficients against the grid value of λ chosen by you. Interpret the graph.

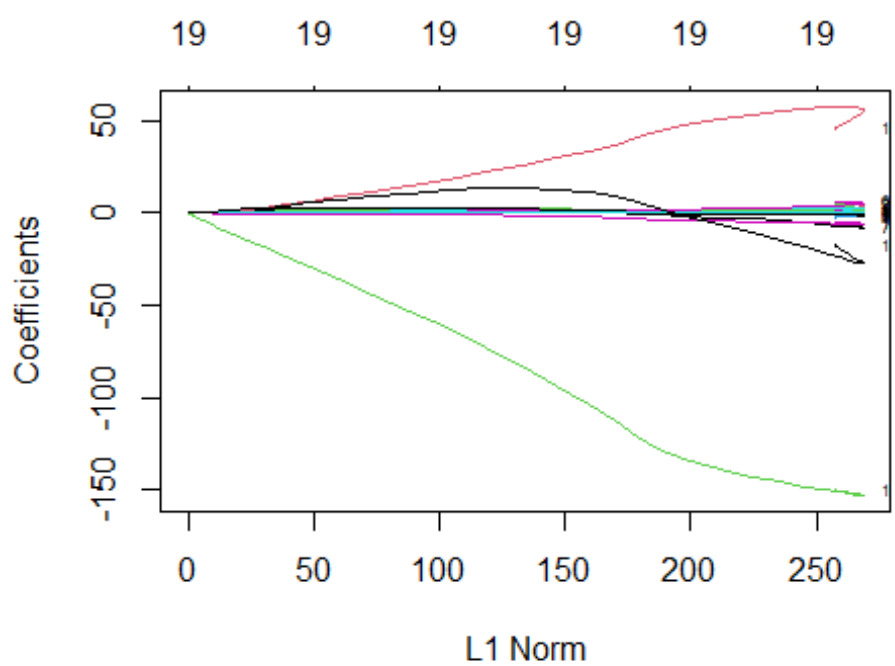
```
ridge_mod = glmnet(x_train, y_train, alpha = 0, lambda = lambda_grid,
standardize = TRUE)
plot(ridge_mod, xvar = "lambda", label = TRUE)
```




```
plot(ridge_mod, xvar = "dev", label = TRUE)
```




```r
plot(ridge_model, xvar = "norm", label = TRUE)
```



X-axis Interpretation:

The X-axis shows the ratio | $|\hat{\beta}_{\text{ridge}}|$ $|2,/,|$ $|\hat{\beta}$ls| $|_2$, which varies between 0 and 1.

At 0 (Left End):

This point corresponds to an extremely large penalty ($\lambda \to \infty$), where all coefficient estimates are shrunk completely to zero.

At 1 (Right End):

This represents the absence of regularization ($\lambda = 0$), meaning the Ridge estimates are exactly the same as the ordinary Least Squares solution.

Direction of Change:

In contrast to the earlier plot (Question 4), where increasing λ (moving right) led to a simpler model, here the model becomes simpler as we move left toward 0, indicating stronger shrinkage.

6. Run the Ridge regression on training data by choosing λ by cross-validation. Find the test MSE for the optimal choice of λ obtained using cross validation.

```
cv_ridge = cv.glmnet(x_train, y_train, alpha = 0)
best_lambda_ridge = cv_ridge$lambda.min
x_test = model.matrix(Salary ~ ., test_data)[, -1]
y_test = test_data$Salary
ridge_pred = predict(cv_ridge, s = best_lambda_ridge, newx = x_test)
mse_ridge = mean((ridge_pred - y_test)^2)
print(mse_ridge)

## [1] 91868.16
```

7. Fit a LASSO model on the training set, with λ chosen by cross-validation. Report the test MSE, along with the number of non-zero coefficient estimates.

```
cv_lasso = cv.glmnet(x_train, y_train, alpha = 1)
best_lambda_lasso = cv_lasso$lambda.min
lasso_pred = predict(cv_lasso, s = best_lambda_lasso, newx = x_test)
mse_lasso = mean((lasso_pred - y_test)^2)
lasso_coef = predict(cv_lasso, type = "coefficients", s = best_lambda_lasso)
print(mse_lasso)

## [1] 94489.8

print(sum(lasso_coef != 0))

## [1] 9
```

8. Fit an Elastic Net model on the training set, with λ1 and λ2 chosen by cross-validation. Report the test MSE. Also report the predictors with non-zero coefficients.

```
alpha_input = 0.5 # For Elastic Net

cv_enat = cv.glmnet(x_train, y_train, alpha = alpha_input)
```

```r
best_lam = cv_enat$lambda.min

enat_pred = predict(cv_enat, s = best_lam, newx = x_test)
mse_enat = mean((y_test - enat_pred)^2)

enat_coef = predict(cv_enat, type = "coefficients", s = best_lam)
enat_coef_mat = as.matrix(enat_coef)
active_coeffs = enat_coef_mat[enat_coef_mat != 0, , drop = FALSE]

print(mse_enat)
```

```
## [1] 93718.42
```

```r
print(active_coeffs)
```

```
##                 s=40.24628
## (Intercept)  100.43320342
## Hits           0.57781704
## RBI            2.35428401
## Walks          1.37878606
## CHits          0.09527549
## CRuns          0.12069283
## CRBI           0.20721690
## LeagueN        2.65634072
## DivisionW   -115.84724474
## PutOuts        0.28989203
```

9. Compare the test MSE 's of least squares regression, Ridge regression, LASSO and ELASTIC NET and comment on how accurately we can predict the Salary of a baseball player.

In this particular run, Ridge Regression achieved the lowest test MSE. This indicates that although the Hitters dataset contains many predictors, most of them likely provide small but meaningful contributions to the response. Because Ridge Regression shrinks coefficients without eliminating them entirely, it is able to retain these subtle effects more effectively than LASSO or Elastic Net, which may discard some useful variables by setting their coefficients to zero.

Predicting a baseball player's salary remains difficult for several reasons:

High Variability: With an MSE in the range of 90,000, the corresponding RMSE (approximately 303) translates to an average prediction error of about $303,000. Considering that many player salaries lie between $100,000 and $1 million, this represents a substantial margin of error.

Non-Linearity: Player salaries tend to follow a skewed or exponential-like distribution rather than a linear pattern. As a result, modeling log(Salary) often produces more reliable predictions than using the raw salary values.

Impact of Regularization: The fact that Ridge, LASSO, and Elastic Net outperform standard Least Squares highlights the presence of overfitting in the Hitters dataset. Introducing a regularization parameter (λ) helps control model complexity and improves its ability to generalize to unseen data.